CIFAR-10 Classification with a Modified ResNet

Zihan Xu
Kaggle team: Zihan Xu
Deep Learning Project 1, Fall 2026

https://github.com/Zihan-Xu/DL-project1

1. Introduction

I trained a modified ResNet-20 for the course CIFAR-10 competition [7]. The final model has 2,436,346 trainable parameters and achieved 87.90% accuracy on the Kaggle public leaderboard. It was trained from scratch on all 50,000 permitted training images. The selected training configuration reached 96.04% mean validation accuracy in a three-seed study before the full-data fit. These are different evaluations: the validation result comes from models trained on 40,000 images, while the public score comes from the final submitted model.

2. Model and training

The network follows the CIFAR ResNet design [1], with three residual stages and three blocks per stage. Each block contains two 3 x 3 convolutions with batch normalization and ReLU, plus a shortcut. I used 48, 96 and 192 channels across the stages, followed by global average pooling and a ten-class linear layer. This adds capacity over the narrow baseline while keeping the parameter count below the five-million limit.

At stage transitions, a fixed 3 x 3 binomial BlurPool filter downsamples both the residual and shortcut paths [2]. The shortcut also uses a learned 1 x 1 projection to match channel counts. BlurPool adds no trainable parameters, although it increases computation. A single forward pass costs about 436.1 million multiply-accumulate operations.

Development used a fixed, class-stratified split of 40,000 training images, 5,000 validation images and 5,000 historical local-test images. Images were normalized using statistics computed from the training portion. Checkpoints were selected by raw validation accuracy, with ties assigned to the earlier epoch. The final model used all 50,000 labeled images, seed 42, and the weights after epoch 240. Its former holdout images therefore cannot provide an independent local test score.

Training used cross-entropy and SGD with learning rate 0.1, momentum 0.9, weight decay 0.0005 and batch size 128. The learning rate followed cosine decay without warmup or restarts [6]. Every image appeared once per epoch. A shuffled sampler repaired missing classes by swapping examples so that each batch contained all ten classes. The final batch contained 80 images, giving 391 updates per epoch.

The augmentations were horizontal flipping, rotation within +/-10 degrees, and Cutout with a 12 x 12 mask [3], each applied with probability 0.5. Rotation used bilinear interpolation and reflection padding. Cutout filled the selected region with normalized zero. Dropout of 0.1 was applied before each block's second convolution [4]. Per-sample residual dropping increased linearly from 0.1/9 to 0.1 across the nine blocks [5]. The model used no pretrained weights or external training data.

3. Experiments

The base configuration used the widened network, BlurPool, Cutout and weight decay of 0.0005. I then compared all eight combinations of rotation (R), dropout (D) and residual dropping (S) at 80 epochs, together with two 240-epoch configurations. Each configuration was trained with seeds 142, 143 and 144, for 30 completed runs. The same 5,000 validation images were used throughout.

For every run, I evaluated both raw inference and horizontal-flip test-time augmentation (TTA) at the same saved checkpoint. TTA averages the softmax probabilities of the original image and its horizontal mirror. It doubles inference computation without adding parameters. The configuration and inference policy with the largest total number of correct validation predictions across the three seeds was selected.

Table 1. Mean validation accuracy (%) across three seeds. A 1 enables the corresponding factor. All rows use 2,436,346 parameters.

R / D / S	Epochs	Raw	Flip TTA
000	80	94.827	95.233
001	80	95.047	95.387
010	80	95.080	95.240
011	80	94.927	95.267
100	80	95.173	95.527
101	80	95.313	95.533
110	80	95.233	95.467
111	80	95.440	95.660
000	240	95.427	95.733
111	240	95.967	96.040

The all-factor model trained for 240 epochs had the highest mean accuracy: 95.967% with raw inference and 96.040% with TTA. Its TTA accuracies were 96.06%, 95.88% and 96.18%, giving a seed standard deviation of 0.151 percentage points. At 240 epochs, adding the three regularizers increased raw mean accuracy by 0.540 points over the base. Extending the all-factor model from 80 to 240 epochs added 0.527 points, at roughly three times the training budget.

The small average gain from TTA was less certain. Its paired gain was 0.073 points, with an unadjusted 95% t interval of [-0.396, 0.543]. None of the 34 planned comparisons passed Holm correction at 0.05. I selected the highest observed mean, but these results do not establish a reliable benefit from every individual component. The three seeds measure training variability on one split, and repeated checkpoint and configuration selection can overstate validation performance.

4. Final result and discussion

After selection, I retrained the chosen configuration from scratch on all 50,000 labeled images and used horizontal-flip TTA for the second submission. The model reached 87.90% public accuracy, compared with 85.00% for the earlier core240/raw submission. The gain was 2.90 percentage points. Since the regularization, training-set size and inference policy changed together, this comparison cannot isolate the contribution of one change. The private leaderboard score is not recorded.

The public score is lower than the development validation mean. Recomputing predictions from the released checkpoint reproduced all 2,000 submitted labels, so the checks did not find an export or ID-order error. Evaluation-data differences and repeated validation selection may contribute to the gap, but the hidden labels do not allow its cause to be established. I would use a fresh validation design before extending the search further.

5. Training record and reproducibility

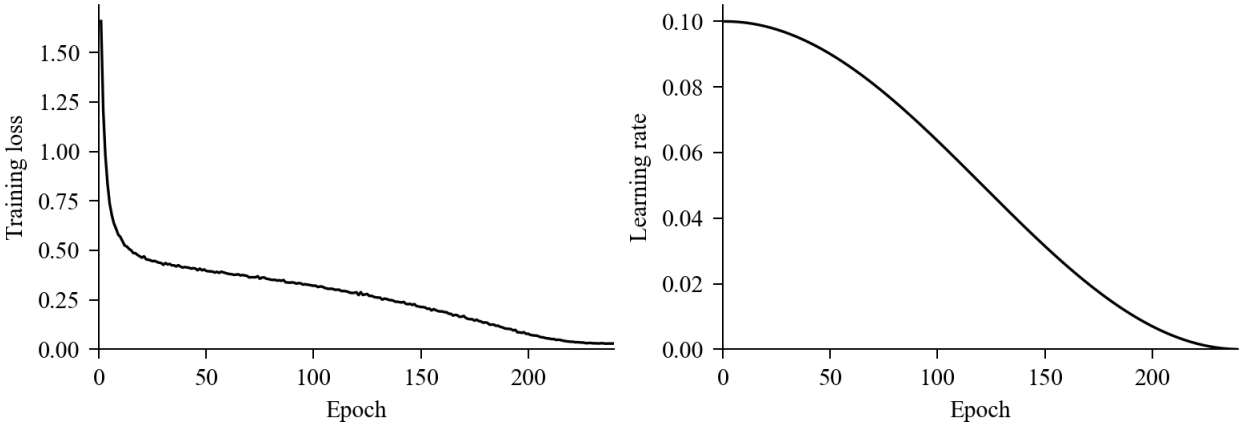


Figure 1. Training loss and learning rate for the final 240-epoch fit. Loss is measured on augmented training images and is not a test metric.

Training was run on one NVIDIA RTX 5070 Ti using Python 3.13.5, PyTorch 2.11.0+cu128 and CUDA 12.8. Training used BF16; prediction used FP32. The complete run contains 93,840 optimizer updates. Normalization was recomputed on the full training set before this fit.

The repository linked on the first page contains the model, runnable training and prediction code, configuration, logs and an executed review notebook. The notebook prints the public accuracy and parameter count and recomputes the validation results in Table 1. To reproduce the submission, place the course data in `data/competition` and run `python predict.py --data-dir data/competition`. Detailed environment setup and verification commands are in the README.

References

[1] K. He, X. Zhang, S. Ren, and J. Sun. Deep Residual Learning for Image Recognition. *CVPR*, 2016. arXiv:1512.03385.

[2] R. Zhang. Making Convolutional Networks Shift-Invariant Again. *ICML*, 2019. arXiv:1904.11486.

[3] T. DeVries and G. W. Taylor. Improved Regularization of Convolutional Neural Networks with Cutout. 2017. arXiv:1708.04552.

[4] N. Srivastava et al. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *JMLR*, 15:1929-1958, 2014. JMLR 15.

[5] G. Huang et al. Deep Networks with Stochastic Depth. *ECCV*, 2016. arXiv:1603.09382.

[6] I. Loshchilov and F. Hutter. SGDR: Stochastic Gradient Descent with Warm Restarts. *ICLR*, 2017. arXiv:1608.03983.

[7] NYU Tandon. *Deep Learning Project 1, Fall 2026.* Course assignment and competition rules. Kaggle rules.